An Algebraic Specification for Quantum Computation in Maude*

Canh Minh Do and Kazuhiro Ogata

Japan Advanced Institute of Science and Technology (JAIST), Nomi, Japan
{canhdo,ogata}@jaist.ac.jp

Abstract. With the rapid advancement of quantum computing, formal verification of quantum systems has become an increasingly important research topic. At the same time, the algebraic specification community, particularly the Maude community, has started to show interest in quantum computing. However, a dedicated algebraic specification framework for quantum computation is still lacking. This paper introduces |AS4QC>, an algebraic specification for quantum computation in Maude, which provides a formal framework for modeling, symbolic and exact reasoning about, and verifying quantum systems. We employ the ring $\mathbb{D}[\omega]$ to exactly represent complex numbers in quantum computation without approximation, use Dirac's bra-ket notation to model quantum states and quantum operations, and leverage a set of laws from quantum mechanics and basis matrix operations expressed in Dirac notation to automate reasoning about quantum computation. As case studies, we use |AS4QC> to verify the correctness of several nontrivial quantum programs, demonstrating the effectiveness and practicality of our approach.

Keywords: algebraic specification · quantum computation · verification

1 Introduction

Quantum computing has attracted significant attention worldwide, and with substantial investment from both governments and major companies, the development of large-scale quantum computers is increasingly seen as a matter of time and sustained effort. Quantum computing differs fundamentally from classical computing by exploiting principles of quantum mechanics, such as superposition, entanglement, and interference. These properties enable quantum algorithms, such as Shor's algorithm for integer factorization and discrete logarithms [24], and Grover's algorithm for unstructured database search [12], to achieve substantial speedups over their best-known classical counterparts. However, due to the counterintuitive and probabilistic nature of quantum computation, the likelihood of programming errors in quantum programs is much higher than in classical ones. Moreover, traditional testing techniques are much less

* This work was supported by JSPS KAKENHI Grant Numbers JP23K28060, JP23K19959, JP24K20757, JP24KK0185.

effective in the quantum setting because of nondeterminism and probabilistic outcomes inherent in quantum measurement. Consequently, formal verification is expected to play a crucial role in ensuring the reliability of quantum systems before they can be trusted in safety-critical and security-sensitive applications.

The algebraic specification community, particularly the Maude community, has started to show interest in quantum computing. However, a dedicated algebraic specification framework for quantum computation is still lacking, which makes the development of formal verification techniques for quantum systems in Maude challenging. In this paper, we present `|AS4QC>`, an algebraic specification for quantum computation in Maude, which provides a formal framework for modeling, symbolic and exact reasoning about, and verifying quantum systems. To develop `|AS4QC>`, we need to support algebraic representations of complex numbers, quantum states, and quantum operations, as well as automated reasoning about quantum computation. Kliuchnikov et al. conjectured in [16], and Giles and Selinger later proved in [11], that a unitary matrix can be exactly realized by a quantum circuit over the universal Clifford + T gate set if and only if all its matrix entries belong to the ring $\mathbb{D}[\omega]$. Therefore, it is promising to study the ring $\mathbb{D}[\omega]$ and its use for the exact algebraic representation of complex numbers arising in quantum computation. Accordingly, we employ $\mathbb{D}[\omega]$ to algebraically represent complex numbers in quantum computation without approximation. For the algebraic representation of quantum states and quantum operations, we adopt Dirac's bra-ket notation [5] instead of explicitly using complex vectors and matrices as in [20,15,27,25]. Dirac notation is widely used in quantum mechanics because of its succinctness and convenience, and it allows us to obtain compact representations that are well-suited to algebraic manipulation. For automated reasoning, we leverage a set of laws from quantum mechanics and basis matrix operations expressed in Dirac notation to perform quantum computation.

Maude [17] is a high-level specification and programming language based on rewriting logic [18], focusing on simplicity, expressiveness, and performance, and providing a wide range of formal analysis facilities. Therefore, we use it to develop the algebraic specification `|AS4QC>` for quantum computation. The algebraic specification allows us to use symbolic values for complex numbers, thereby enabling symbolic quantum computation, which is a major advantage of our approach. As case studies, we use `|AS4QC>` to verify the correctness of several nontrivial quantum programs, including Entanglement, Quantum Teleportation [1], Entanglement Swapping [28], Quantum Secret Sharing [14], Quantum Relay Scheme [4], Bidirectional Quantum Teleportation [13], Quantum Network Coding [21], and Grover's Search [12], to demonstrate the effectiveness and practicality of our approach. The algebraic specification `|AS4QC>` and case studies are publicly available at https://github.com/canhminhdo/as4qc.

The rest of the paper is organized as follows. Section 2 introduces basic notations from quantum computation. Section 3 presents the algebraic specification for quantum computation. Section 4 describes a case study in `|AS4QC>`. Section 5 shows the experimental results for several case studies. Section 6 mentions related work and finally concludes the paper with some future directions.

2 Preliminaries

This section describes basic notations from quantum computation (refer to [19] for more details) and assumes that the reader has some basic knowledge of linear algebra.

The state space of a quantum system is a Hilbert space $\mathcal{H}$, which is a complete complex vector space equipped with an inner product. A pure state of a quantum system is described by a unit vector written in Dirac's ket notation as $|\psi\rangle \in \mathcal{H}$. The conjugate transpose of $|\psi\rangle$ is a row vector written in Dirac's bra notation as $\langle\psi| \triangleq |\psi\rangle^\dagger$. The inner product of $|\psi\rangle$ and $|\phi\rangle$ is written as $\langle\psi|\phi\rangle$ and they are orthogonal if $\langle\psi|\phi\rangle = 0$. The outer product of them, denoted $|\psi\rangle\langle\phi|$, is a linear operator that maps any $|\psi'\rangle \in \mathcal{H}$ to $\langle\phi|\psi'\rangle\,|\psi\rangle$. The length of $|\psi\rangle$ is defined as $\|\,|\psi\rangle\,\| \triangleq \sqrt{\langle\psi|\psi\rangle}$. A set of vectors $B \triangleq \{|i\rangle : i \in I\} \in \mathcal{H}$ is orthonormal if each $|i\rangle$ is normalized and every two vectors in the set are orthogonal. Furthermore, if they span the whole space $\mathcal{H}$; that is, any vector in $\mathcal{H}$ can be written as a linear combination of vectors in B, then B is called an orthonormal basis of $\mathcal{H}$.

A qubit is a quantum system whose state space is the two-dimensional Hilbert space $\mathcal{H}_2 = \mathbb{C}^2$. The state of a qubit can be written as $\alpha\,|0\rangle + \beta\,|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$, where $\alpha, \beta \in \mathbb{C}$, $|0\rangle \triangleq [1\ 0]^T$, and $|1\rangle \triangleq [0\ 1]^T$ with ^{T} denoting the transpose operator. The coefficients α and β are called the amplitudes of this quantum state. The set $\{|0\rangle, |1\rangle\}$ forms an orthonormal basis of $\mathcal{H}_2$ and is called the computational basis. For multiple qubits, we use tensor products of Hilbert spaces. Let $\mathcal{H}_A$ and $\mathcal{H}_B$ be the Hilbert spaces of systems A and B, respectively. The tensor product of $\mathcal{H}_A$ and $\mathcal{H}_B$ is the Hilbert space $\mathcal{H}_A \otimes \mathcal{H}_B$ (or $\mathcal{H}_{AB}$) of their composite system AB. It consists of linear combinations of product states $|\psi\rangle \otimes |\phi\rangle$ (often written $|\psi\phi\rangle$ or $|\psi\rangle\,|\phi\rangle$) with $|\psi\rangle \in \mathcal{H}_A$ and $|\phi\rangle \in \mathcal{H}_B$. Any state in $\mathcal{H}_A \otimes \mathcal{H}_B$ that cannot be written as a product state is called an entangled state. For example, the Bell (or EPR) state $|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$ is maximally entangled.

In quantum computing, we mainly perform two types of operations: unitary transformation (also known as quantum gates) and quantum measurement. A unitary transformation on a quantum system in the Hilbert space $\mathcal{H}$ is a linear operator $U : \mathcal{H} \to \mathcal{H}$ satisfying $UU^\dagger = U^\dagger U = I_{\mathcal{H}}$, where $U^\dagger$ is the conjugate transpose of U and $I_{\mathcal{H}}$ is the identity operator on $\mathcal{H}$. After unitary transformation, a state $|\psi\rangle$ is changed to $U\,|\psi\rangle$ deterministically. We present the definition of some standard unitary transformations as follows.

$$\mathrm{X} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \mathrm{Y} = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \qquad \mathrm{Z} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix},$$

$$\mathrm{H} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \qquad \mathrm{T} = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}, \qquad \mathrm{CX} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

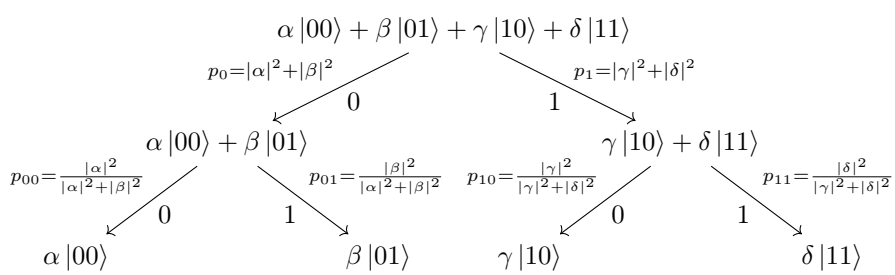
A quantum measurement on a quantum system in the Hilbert space $\mathcal{H}$ is a collection of measurement operators $\{M_m\}$ satisfying the completeness relation

$\sum_m M_m^\dagger M_m = I$. Measuring a state $|\psi\rangle$ yields outcome m with probability $p(m) = \|M_m |\psi\rangle\|^2$, and the state is changed to $|\psi_m\rangle = \frac{M_m|\psi\rangle}{\sqrt{p(m)}}$. Note that $M_m |\psi\rangle$ is normalized by dividing by its norm $\sqrt{p(m)} = ||M_m |\psi\rangle||$, so that it becomes a unit vector. As an example, we consider the measurement $\{M_0 = |0\rangle\langle 0|, M_1 = |1\rangle\langle 1|\}$ applied to the state $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$. The probabilities of obtaining outcomes 0 and 1 are $|\alpha|^2$ and $|\beta|^2$, respectively and the state collapses to $|0\rangle$ or $|1\rangle$ accordingly. Consequently, quantum measurement changes a quantum system in a nondeterministic and probabilistic way.

The measurement involving multiple qubits is more complex. Let us consider a two-qubit system in a state $|\psi\rangle = \alpha |00\rangle + \beta |01\rangle + \gamma |10\rangle + \delta |11\rangle$ with $|\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1$. We use the measurement $\{M_0 = |0\rangle\langle 0|, M_1 = |1\rangle\langle 1|\}$, but note that we do not normalize quantum states after each measurement. When we measure the first qubit, one of the following will occur:

- the outcome 0 with probability of $p_0 = |\alpha|^2 + |\beta|^2$, collapsing the quantum state to $\alpha |00\rangle + \beta |01\rangle$, and
- the outcome 1 with probability of $p_1 = |\gamma|^2 + |\delta|^2$, collapsing the quantum state to $\gamma |10\rangle + \delta |11\rangle$.

Next, we measure the second qubit. The possible outcomes are illustrated in the following diagram.



In this diagram, nodes represent quantum states, and transitions are labeled with the probabilities and measurement outcomes. The total probabilities of obtaining 00, 01, 10, and 11 as the result of the two measurements are $p_0 p_{00} = |\alpha|^2$, $p_0 p_{01} = |\beta|^2$, $p_1 p_{10} = |\gamma|^2$, and $p_1 p_{11} = |\delta|^2$. It is common to normalize quantum states to unit vectors after each measurement, ensuring that the sum of the absolute squares of the amplitudes is one. However, the diagram shows that a different normalization is more convenient. This paper adopts the normalization convention from [22,8] such that each state is normalized so that the sum of the absolute squares of the amplitudes equals the total probability of reaching that state. It means that when measuring a state $|\psi\rangle$ with outcome m, the state changes to $|\psi_m\rangle = M_m |\psi\rangle$, which may not be a unit vector since normalization (division by its norm) is omitted. This convention significantly simplifies our representation and computation and does not affect physical observations.

For each closed subspace V of $\mathcal{H}$, there exists a unique projection operator $\mathcal{P}_V$. Every state $|\psi\rangle \in \mathcal{H}$ can be written as $|\psi\rangle = |\psi_V\rangle + |\psi_{V^\perp}\rangle$ with $|\psi_V\rangle \in V$

and $|\psi_{V^\perp}\rangle \in V^\perp$, the orthogonal complement of V. The projection $\mathcal{P}_V : \mathcal{H} \to V$ is defined by $\mathcal{P}_V |\psi\rangle = |\psi_V\rangle$. Since closed subspaces of $\mathcal{H}$ are in one-to-one correspondence with projection operators, we often identify $\mathcal{P}_V$ and V, and write $\mathcal{P}_V$ as $\mathcal{P}$ when the closed subspace V is clear from context. Moreover, if a state $|\psi\rangle$ is in the closed subspace V of the projection operator $\mathcal{P}_V$, then $\mathcal{P}_V |\psi\rangle = |\psi\rangle$.

3 Algebraic Specification for Quantum Computation

This section presents the algebraic specification `|AS4QC>` for quantum computation in Maude, including complex number representation, quantum states and quantum operations, automated reasoning about quantum computation based on a set of laws from quantum mechanics and basic matrix operations, and the usability of `|AS4QC>`. We assume that the reader is familiar with Maude.

3.1 Complex Number Representation

This work focuses on symbolic and exact reasoning for quantum computation. Therefore, it is necessary to exactly represent complex numbers arising in quantum computation without approximation. Kliuchnikov et al. conjectured in [16], and Giles and Selinger later proved in [11], that a unitary matrix can be exactly realized by a quantum circuit over the universal Clifford + T gate set, possibly using one additional ancilla, if and only if its matrix entries belong to the ring $\mathbb{D}[\omega]$. This ring consists of all complex numbers of the form $\frac{1}{\sqrt{2}^k}(a + b\omega + c\omega^2 + d\omega^3)$, where $k \in \mathbb{N}, a, b, c, d \in \mathbb{Z}$, and $\omega = e^{i\pi/4} = \cos(\pi/4) + i\,\sin(\pi/4) = (1 + i)/\sqrt{2}$. Using the ring $\mathbb{D}[\omega]$, a complex number is algebraically represented by a quadruple (a, b, c, d) of integers and a normalization factor k of a natural number. Although $\mathbb{D}[\omega]$ does not include all complex numbers, it forms a dense subset of complex numbers such that any complex number can be approximated to arbitrary precision by an element of the ring $\mathbb{D}[\omega]$ [29]. Moreover, the subset of complex numbers is sufficient to describe various standard quantum gates. Our algebraic specification currently supports X, Y, Z, H, S, T, Rx[$\pi/2$], Ry[$\pi/2$], Rz[$\pi/2$], CX, CY, CZ, SWAP, CCX, CCY, CCZ, MCX, MCY, MCZ, MCSWAP, which already includes the universal Clifford + T gate set. This algebraic representation of complex numbers is also sufficient to exactly represent all reachable states of OpenQASM circuits [1] when the initial state is a basis state because complex numbers in the ring $\mathbb{D}[\omega]$ are closed under addition and multiplication.

We employ the ring $\mathbb{D}[\omega]$ to algebraically represent complex numbers and also assume that all amplitudes of a quantum state share a common normalization factor k as in [3,2]. Rather than storing this factor for each amplitude, we store a single global value k for the whole quantum state. The value of k can be inferred from the fact that the sum of probabilities over all basis states is one. Moreover, the initial quantum state is usually a basis state, k is initialized to zero in most

[1] OpenQASM (open quantum assembly language) is an imperative programming language for describing quantum circuits used by IBM.

cases. For each application of an H, Rx[$\pi/2$], Ry[$\pi/2$], or Rz[$\pi/2$] gate, the value of k is incremented since these gates introduce a factor of $1/\sqrt{2}$. It is important to note that this factor is omitted in our representations for these quantum gates, which further simplifies their algebraic representations. After all operations have been applied, the final quantum state is normalized by multiplying it by $(1/\sqrt{2})^k$. Under this representation, each amplitude of a quantum state is represented as a quadruple (a, b, c, d) of integers and a value k is stored globally. Arithmetic operations on complex numbers in this quadruple form, such as addition and multiplication, are therefore significantly simplified.

We have the following lemma to distinguish between zero and nonzero complex numbers in our algebraic specification.

Lemma 1 *Let* $\frac{1}{\sqrt{2}^k}(a + b\omega + c\omega^2 + d\omega^3)$ *be an element of the ring* $\mathbb{D}[\omega]$*, where* $k \in \mathbb{N}$*,* $a, b, c, d \in \mathbb{Z}$*, and* $\omega = e^{i\pi/4}$*. Then, we have*

$$\frac{1}{\sqrt{2}^k}(a + b\omega + c\omega^2 + d\omega^3) = 0 \text{ if and only if } a = b = c = d = 0.$$

Proof. It is straightforward to prove this lemma by substituting $\omega = (1 + i)/\sqrt{2}$*,* $\omega^2 = i$*, and* $\omega^3 = (i - 1)/\sqrt{2}$*.*

Based on the algebraic representation of complex numbers in the ring $\mathbb{D}[\omega]$ and Lemma 1, we formalize complex numbers as quadruples of integers and declare addition, multiplication, and conjugation operations for complex numbers in Maude as follows.

```
sorts ZeroCpx NzCpx Cpx .
subsorts ZeroCpx NzCpx < Cpx .
op (_,_,_,_) : Zero Zero Zero Zero -> ZeroCpx [ctor] .
op (_,_,_,_) : NzInt Int Int Int   -> NzCpx   [ctor] .
op (_,_,_,_) : Int NzInt Int Int   -> NzCpx   [ctor] .
op (_,_,_,_) : Int Int NzInt Int   -> NzCpx   [ctor] .
op (_,_,_,_) : Int Int Int NzInt   -> NzCpx   [ctor] .
op (_,_,_,_) : Int Int Int Int     -> Cpx     [ctor] .
op _+_ : Cpx Cpx     -> Cpx   [comm assoc prec 33] .
op _*_ : Cpx Cpx     -> Cpx   [comm assoc prec 31] .
op _*_ : NzCpx NzCpx -> NzCpx [ditto] .
op _^* : Cpx   -> Cpx   [prec 29] .
op _^* : NzCpx -> NzCpx [prec 29] .
op _^* : NzCpx -> NzCpx [prec 29] .
```

where `ZeroCpx`, `NzCpx`, and `Cpx` denote the sorts of zero complex numbers, nonzero complex numbers, and complex numbers, respectively. The addition, multiplication, and conjugation operations of complex numbers of the form $a + b\omega + c\omega^2 + d\omega^3$ become simple by substituting ω, ω^2, ω^3, $\omega^4 = -1$, $\omega^* = -\omega^3$, $(\omega^2)^* = -\omega^2$, and $(\omega^3)^* = -\omega$ as appropriate and performing the corresponding calculations. This process finally yields concise algebraic results. For example, the conjugate $(a + b\omega + c\omega^2 + d\omega^3)^*$ is $(a - d\omega - c\omega^2 - a\omega^3)$. Accordingly, we define the conjugation operation on complex numbers in Maude as follows.

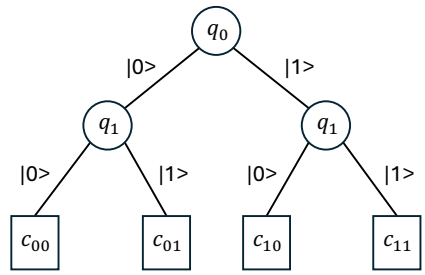


Fig. 1: The binary decision tree of $c_{00}\,|00\rangle + c_{01}\,|01\rangle + c_{10}\,|10\rangle + c_{11}\,|11\rangle$

```
eq (I1, I2, I3, I4)^* = (I1, - I4, - I3, - I2) .
```

where `I1`, `I2`, `I3`, `I4` are variables of sort `Int`. One important operation on complex numbers in quantum computation is computing the squared absolute value of a complex number, which is used to determine the probability of a basis state in a quantum state. We present the following result to facilitate the calculation of the probability of a basis state.

$$\begin{aligned}|a + b\omega + c\omega^2 + d\omega^3|^2 &= (a + b\omega + c\omega^2 + d\omega^3)(a + b\omega + c\omega^2 + d\omega^3)^* \\ &= (a + b\omega + c\omega^2 + d\omega^3)(a - d\omega - c\omega^2 - a\omega^3) \\ &= a^2 + b^2 + c^2 + d^2 + \sqrt{2}(ab + bc + cd - ad).\end{aligned}$$

Note that the value of the probability is a real number when $\sqrt{2}$ appears in the result of $|a + b\omega + c\omega^2 + d\omega^3|^2$.

3.2 Quantum States and Quantum Operations

Dirac's bra-ket notation [5] is widely used in quantum mechanics because of its succinctness and convenience. Therefore, we adopt it to represent quantum states and quantum operations, rather than explicitly using complex vectors and matrices. This idea is inherited from our previous work [6,9]. This notation makes our representation compact and well-suited to algebraic manipulation.

We first formalize basic vectors, covectors, and matrices written in Dirac's bra-ket notation as $|0\rangle$ and $|1\rangle$, $\langle 0|$ and $\langle 1|$, and $|0\rangle\langle 0|$, $|0\rangle\langle 1|$, $|1\rangle\langle 0|$, and $|1\rangle\langle 1|$, respectively. For each of vectors, covectors, and matrices, we formalize constructors of scalar multiplication $\cdot$, addition $+$, multiplication $\times$, tensor product $\otimes$, and conjugate transpose †. Additionally, we formalize the inner product $\langle _|_\rangle$ and norm square $\|_\|^2$ operations for vectors. Terms representing quantum states and quantum operations are then built from scalars (or complex numbers) formalized in Section 3.1, basic elements, and their constructors.

Quantum states over n qubits can be viewed as a binary decision tree over n quantum variables. For example, a two-qubit state $c_{00}\,|00\rangle + c_{01}\,|01\rangle + c_{10}\,|10\rangle + c_{11}\,|11\rangle$ can be represented by a binary decision tree over the quantum variables

q_0 and q_1 as shown in Figure 1. This representation is inspired by [30,3] and the probability amplitude c_{00} of the basis state $|01\rangle$ corresponds to the branch $q_0 \xrightarrow{|0\rangle} q_1 \xrightarrow{|1\rangle} c_{01}$ in the binary decision tree. Although a two-qubit state can be written as a linear combination of basis states, this representation is not efficient for computation. Therefore, we adopt a binary decision tree-based representation, expressing the state as $|0\rangle \otimes (c_{00} \cdot |0\rangle + c_{01} \cdot |1\rangle) + |1\rangle \otimes (c_{10} \cdot |0\rangle + c_{11} \cdot |1\rangle)$. This hierarchical structure simulates a binary decision tree, uniquely represents quantum states, and enables more efficient manipulation. For example, if we measure the first qubit q_0, we only need to consider the term $|0\rangle \otimes (c_{00} \cdot |0\rangle + c_{01} \cdot |1\rangle)$ and simply discard the remainder in the case of a zero outcome; and similarly for the case of a one outcome. Therefore, we use this binary decision tree-based representation for quantum states in our algebraic specification. Note that this representation is also used for the bra representations of quantum states.

Quantum operations are built from scalars and basic matrices using their constructors. We do not impose a normal form representation for quantum operations as we have done for quantum states; instead, we prepare them in a designated representation to speed up computation when applying quantum operations to quantum states. This is possible because users do not directly manipulate quantum gates but interact with them through a high-level interface provided by our algebraic specification. We can definitely make a normal form for quantum operations for certain purposes, such as equivalence checking of quantum circuits; however, doing so may degrade performance, and therefore, we do not adopt a normal form for quantum operations in this work. For intuition, some standard unitary matrices can be represented as follows.

$$
\begin{aligned}
\mathrm{X} &= |0\rangle\langle 1| + |1\rangle\langle 0|\,, & \mathrm{Y} &= (0,0,-1,0) \cdot |0\rangle\langle 1| + (0,0,1,0) \cdot |1\rangle\langle 0|\,,\\
\mathrm{Z} &= |0\rangle\langle 0| + (-1,0,0,0) \cdot |1\rangle\langle 1|\,, & \mathrm{H} &= |0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| + (-1,0,0,0) \cdot |1\rangle\langle 1|\,,\\
\mathrm{CX} &= |0\rangle\langle 0| \otimes \mathrm{I} + |1\rangle\langle 1| \otimes \mathrm{X}, & \mathrm{T} &= |0\rangle\langle 0| + (0,1,0,0) \cdot |1\rangle\langle 1|\,,\\
\mathrm{P0} &= |0\rangle\langle 0|\,, & \mathrm{P1} &= |1\rangle\langle 1|\,,
\end{aligned}
$$

where the factor $1/\sqrt{2}$ is omitted in the Hadamard gate H, as explained in Section 3.1 and the symbol I denotes the identity operator on the Hilbert space $\mathcal{H}_2$. We also present the projection operators P0 and P1 corresponding to the measure operators M_0 and M_1, respectively, as shown in Section 2. The algebraic representation of complex numbers in the ring $\mathbb{D}[\omega]$ again simplifies the representation of these quantum operations, especially in the cases of H and T gates. Note that X, Y, Z, H, and T gates and P0 and P1 projectors act on one qubit, while CX gate acts on two qubits. Let $\bar{q} = q_0, \ldots, q_{n-1}$ be a quantum register, that is, a sequence of distinct quantum variables, used to refer to quantum states $|\psi\rangle$ over n qubits. Let $\bar{r}_i$ be a subsequence of $\bar{q}$. To apply a quantum gate U_i to a quantum state $|\psi\rangle$ with respect to the quantum register $\bar{r}_i$, we first construct the cylindrical extension of U_i in the form $\overline{U}_i = U_i \otimes I_i$ so that the dimension of $\overline{U}_i$ matches that of $|\psi\rangle$. Here, I_i is the identity operator on the Hilbert space $\mathcal{H}_{\bar{q} \setminus \bar{r}_i}$. For example, when applying the Hadamard gate H to the qubit q_0 of a two-qubit state shown in Figure 1, the cylindrical extension of H is $\mathrm{H} \otimes \mathrm{I}$. We use the rep-

resentation of $\mathrm{H} \otimes \mathrm{I}$ in the form $(|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| + (-1,0,0,0) \cdot |1\rangle\langle 1|) \otimes \mathrm{I}$ instead of $|0\rangle\langle 0| \otimes \mathrm{I} + |0\rangle\langle 1| \otimes \mathrm{I} + |1\rangle\langle 0| \otimes \mathrm{I} + (-1,0,0,0) \cdot |1\rangle\langle 1| \otimes \mathrm{I}$, where the tensor product $\otimes$ is distributed over the addition $+$ in the latter representation. We utilize the former representation for quantum operations to make them succinct and improve computational efficiency, which is demonstrated in the next section.

3.3 Automated Reasoning for Quantum Computation

We leverage a set of laws from quantum mechanics and basic matrix operations, as shown in Table 1, to enable automated reasoning for quantum computation in Maude. This set of laws is also inherited from our previous work [6,9]. Because $|0\rangle$ and $|1\rangle$ can be viewed as 2×1 matrices and $\langle 0|$ and $\langle 1|$ can be viewed as 1×2 matrices, these laws effectively describe matrix calculations using Dirac notation, along with zero matrices, identity matrices, and scalars. These laws are formalized as equations in Maude and are used to simplify terms representing quantum states, quantum operations, and the application of quantum operations to quantum states. It is important to note that these laws need to be applied flexibly to achieve the representations of quantum states and quantum operations described in Section 3.2.

For example, we would like to reduce the term $\mathrm{CX} \times ((\mathrm{H} \otimes \mathrm{I}) \times |0\rangle \otimes |0\rangle)$ to check whether its result is $|0\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle$. Note that the factor $1/\sqrt{2}$ in the Hadamard gate is handled by the global value k as explained in Section 3.1. The term says that the H gate acts on the first qubit, followed by the CX gate, where the control and target qubits are the first and second qubits, respectively. The simplification of the term goes as follows:

$\mathrm{H} \times |0\rangle$
$\to (|0\rangle\langle 0| + |0\rangle\langle 1| + |1\rangle\langle 0| + (-1,0,0,0) \cdot |1\rangle\langle 1|) \times |0\rangle$
$\to |0\rangle\langle 0| \times |0\rangle + |0\rangle\langle 1| \times |0\rangle + |1\rangle\langle 0| \times |0\rangle + ((-1,0,0,0) \cdot |1\rangle\langle 1|) \times |0\rangle$
$\to |0\rangle + |1\rangle$

$(\mathrm{H} \otimes \mathrm{I}) \times (|0\rangle \otimes |0\rangle)$
$\to (\mathrm{H} \times |0\rangle) \otimes (\mathrm{I} \times |0\rangle)$
$\to (|0\rangle + |1\rangle) \otimes |0\rangle$
$\to |0\rangle \otimes |0\rangle + |1\rangle \otimes |0\rangle$

$\mathrm{CX} \times ((\mathrm{H} \otimes \mathrm{I}) \times (|0\rangle \otimes |0\rangle))$
$\to (|0\rangle\langle 0| \otimes \mathrm{I} + |1\rangle\langle 1| \otimes \mathrm{X}) \times (|0\rangle \otimes |0\rangle + |1\rangle \otimes |0\rangle)$
$\to (|0\rangle\langle 0| \otimes \mathrm{I} \times |0\rangle \otimes |0\rangle) + (|0\rangle\langle 0| \otimes \mathrm{I} \times |1\rangle \otimes |0\rangle) + (|1\rangle\langle 1| \otimes \mathrm{X} \times |0\rangle \otimes |0\rangle) + (|1\rangle\langle 1| \otimes \mathrm{X} \times |1\rangle \otimes |0\rangle)$
$\to |0\rangle \otimes (\mathrm{I} \times |0\rangle) + |1\rangle \otimes (\mathrm{X} \times |0\rangle)$
$\to |0\rangle \otimes |0\rangle + |1\rangle \otimes ((|1\rangle\langle 0| + |0\rangle\langle 1|) \times |0\rangle)$
$\to |0\rangle \otimes |0\rangle + |1\rangle \otimes (|1\rangle\langle 0| \times |0\rangle + |0\rangle\langle 1| \times |0\rangle)$
$\to |0\rangle \otimes |0\rangle + |1\rangle \otimes |1\rangle$

where multiple rewrite steps are subsumed to avoid verbosity. Using these laws, the application of quantum operations to quantum states can be simplified into new quantum states. The entire process is carried out automatically in Maude with `|AS4QC>`, and the resulting state is the same as expected. The key idea is to reduce the matrix multiplication in the form of $\langle i|j\rangle$ with $i, j \in \{0, 1\}$ into a

Table 1: A set of laws used for automated reasoning about quantum computation

No.	Law
L1	$\langle 0\|0\rangle = \langle 1\|1\rangle = 1, \langle 0\|1\rangle = \langle 1\|0\rangle = 0$
L2	Associativity of $\times, +, \otimes$ and Commutativity of $+$
L3	$0 \cdot \boldsymbol{A}_{m\times n} = \mathbf{O}_{m\times n},\ c \cdot \mathbf{O} = \mathbf{O},\ 1 \cdot \boldsymbol{A} = \boldsymbol{A}$
L4	$c \cdot (\boldsymbol{A} + \boldsymbol{B}) = c \cdot \boldsymbol{A} + c \cdot \boldsymbol{B}$
L5	$c_1 \cdot \boldsymbol{A} + c_2 \cdot \boldsymbol{A} = (c_1 + c_2) \cdot \boldsymbol{A}$
L6	$c_1 \cdot (c_2 \cdot \boldsymbol{A}) = (c_1 \cdot c_2) \cdot \boldsymbol{A}$
L7	$(c_1 \cdot \boldsymbol{A}) \times (c_2 \cdot \boldsymbol{B}) = (c_1 \cdot c_2) \cdot (\boldsymbol{A} \times \boldsymbol{B})$
L8	$\boldsymbol{A} \times (c \cdot \boldsymbol{B}) = (c \cdot \boldsymbol{A}) \times \boldsymbol{B} = c \cdot (\boldsymbol{A} \times \boldsymbol{B})$
L9	$\boldsymbol{A} \otimes (c \cdot \boldsymbol{B}) = (c \cdot \boldsymbol{A}) \otimes \boldsymbol{B} = c \cdot (\boldsymbol{A} \otimes \boldsymbol{B})$
L10	$\mathbf{O}_{m\times n} \times \boldsymbol{A}_{n\times p} = \boldsymbol{A}_{m\times n} \times \mathbf{O}_{n\times p} = \mathbf{O}_{m\times p}$
L11	$\boldsymbol{I}_m \times \boldsymbol{A}_{m\times n} = \boldsymbol{A}_{m\times n} \times \boldsymbol{I}_n = \boldsymbol{A}_{m\times n}$
L12	$\boldsymbol{A} + \mathbf{O} = \mathbf{O} + \boldsymbol{A} = \mathbf{O}$
L13	$\mathbf{O}_{m\times n} \otimes \boldsymbol{A}_{p\times q} = \boldsymbol{A}_{p\times q} \otimes \mathbf{O}_{m\times n} = \mathbf{O}_{mp\times nq}$
L14	$\boldsymbol{A} \times (\boldsymbol{B} + \boldsymbol{C}) = \boldsymbol{A} \times \boldsymbol{B} + \boldsymbol{A} \times \boldsymbol{C}$
L15	$(\boldsymbol{A} + \boldsymbol{B}) \times \boldsymbol{C} = \boldsymbol{A} \times \boldsymbol{C} + \boldsymbol{B} \times \boldsymbol{C}$
L16	$(\boldsymbol{A} \otimes \boldsymbol{B}) \times (\boldsymbol{C} \otimes \boldsymbol{D}) = (\boldsymbol{A} \times \boldsymbol{C}) \otimes (\boldsymbol{B} \times \boldsymbol{D})$
L17	$\boldsymbol{A} \otimes (\boldsymbol{B} + \boldsymbol{C}) = \boldsymbol{A} \otimes \boldsymbol{B} + \boldsymbol{A} \otimes \boldsymbol{C}$
L18	$(\boldsymbol{A} + \boldsymbol{B}) \otimes \boldsymbol{C} = \boldsymbol{A} \otimes \boldsymbol{C} + \boldsymbol{B} \otimes \boldsymbol{C}$
L19	$(c \cdot \boldsymbol{A})^\dagger = c^* \cdot \boldsymbol{A}^\dagger,\ (\boldsymbol{A} \times \boldsymbol{B})^\dagger = \boldsymbol{B}^\dagger \times \boldsymbol{A}^\dagger$
L20	$(\boldsymbol{A} + \boldsymbol{B})^\dagger = \boldsymbol{A}^\dagger + \boldsymbol{B}^\dagger,\ (\boldsymbol{A} \otimes \boldsymbol{B})^\dagger = \boldsymbol{A}^\dagger \otimes \boldsymbol{B}^\dagger$
L21	${\boldsymbol{I}_m}^\dagger = \boldsymbol{I}_m, \boldsymbol{O}^\dagger_{m\times n} = \boldsymbol{O}_{n\times m}, (\boldsymbol{A}^\dagger)^\dagger = \boldsymbol{A}$
L22	$\|0\rangle^\dagger = \langle 0\|,\ \langle 0\|^\dagger = \|0\rangle,\ \|1\rangle^\dagger = \langle 1\|,\ \langle 1\|^\dagger = \|1\rangle$

scalar and simplify the matrix representation by absorbing ones and eliminating zeros as soon as possible (see the law with label L3). For example, given the term $|1\rangle\langle 1| \otimes \mathrm{X} \times |0\rangle \otimes |0\rangle$, we first check that $|1\rangle\langle 1| \times |0\rangle = 0$ and then it is unnecessary to consider $\mathrm{X} \times |0\rangle$ and return the zero vector immediately. This rewrite strategy significantly improves performance when matrices and vectors corresponding to multiple qubits are used instead of X and $|0\rangle$ as in this simple example. Moreover, the representation of quantum states described in Section 3.1 is specifically designed to work with this strategy, thereby avoiding many unnecessary computations. In this way, automated reasoning about quantum computation using Dirac's bra-ket notation can be achieved through rewriting in Maude.

3.4 Usability of |AS4QC>

We present the usability of `|AS4QC>` to perform automated reasoning about quantum computation in Maude. In our algebraic specification, we define the sorts `Vect`, `CoVect`, and `Mat` to represent vectors, covectors, and matrices,

which correspond to quantum states, bra representations of quantum states, and quantum operations, respectively. The scalar multiplication $\cdot$, addition $+$, multiplication $\times$, tensor product $\otimes$, and conjugate transpose † are formalized in Maude as the operators `_._`, `_+_`, `_x_`, `_(x)_`, `_^+`, respectively. The inner product $\langle _|_ \rangle$ and norm square $\|_\|^2$ operations for vectors are formalized as the operators `<_,_>` and `||_||^2`, respectively. Recall the complex number representation in Section 3.1, we formalize it as the operator `(_,_,_,_,_)`. Note that underscores denote operator parameters.

We support the following quantum operations in Maude for reasoning about quantum computation.

```
sort Gate .
subsort Gate < Mat .
--- single-qubit operations
ops P0 P1 X Y Z H S T Rx Ry Rz : Nat -> Gate .
--- two-qubit operations
ops CX CY CZ SWAP : Nat Nat -> Gate .
--- three-qubit operations
ops CCX CCY CCZ CSWAP : Nat Nat Nat -> Gate .
--- multi-qubit operations
ops MCX MCY MCZ : NatList Nat     -> Gate .
op  MCSWAP      : NatList Nat Nat -> Gate .
```

The sort `Gate` is defined as a subsort of `Mat` to represent quantum operations. The parameters of quantum operations are natural numbers denoting the qubits to which they are applied. For multi-qubit operations, a list of natural numbers is used to specify the control qubits, together with additional parameters of natural numbers denoting the target qubits. The application of quantum operations is carried out automatically through simplification, as described in Section 3.3.

We next define a computation in `|AS4QC>` that initially takes a list of quantum gates and a quantum state given as input as follows.

```
sort Comp .
op [_,_]      : GateList Vect         -> Comp .
op [_,_,_,_]  : GateList Vect Nat Nat -> Comp .
op [_,_]      : Vect Float            -> Comp .
```

The sort `GateList` represents a list of quantum operations, and the sort `Comp` represents computations. Given a list `GL` of quantum operations of sort `GateList` and a quantum state `V` of sort `Vect`, we first compute the number of qubits and initialize the global normalization factor k to zero using the following equation. Since the number of qubits is frequently used to construct cylindrical extensions of quantum operations, it is computed once at the beginning and reused later.

```
eq [GL, V] = [GL, V, sizeV(V), 0] .
```

The quantum operations in `GL` are applied to `V` in order from left to right until `GL` becomes `nil`, indicating that no further quantum operations remain. At this point, the final output is obtained and can be analyzed. To handle the nondeterministic nature of quantum measurement, we formalize a set of computations

with the sort `CompSet` and formalize the nondeterministic choices of quantum operations with the operator `_U_` as follows.

```
op _U_ : GateList GateList -> GateList .
eq [(GL1 U GL2) GL, V, N, N']
 = [GL1 GL, V, N, N'], [GL2 GL, V, N, N'] .
```

where `GL1`, `GL2`, and `GL` are variables of sort `GateList`, `V` is a variable of sort `Vect`, `N` and `N'` are variables of sort `Nat`, and the operator `_,_` is the constructor of the set of computations. The equation states that both choices are handled as two separate computations.

To support analysis of the final output, we provide the following operations.

```
op analyze  : CompSet -> CompSet .
op analyze  : CompSet Vect -> CompSet .
op check    : CompSet Formula -> CompSet .
```

Given a set of final computations, the operator `analyze` returns a set of computations where each computation now consists of the final quantum state and its reaching probability. If a basis state is provided as a parameter, each computation consists of the final quantum state and the probability of that basis state in the final quantum state. Given a set of final computations and a formula, the operator `check` returns a set of computations that do not satisfy the formula. The formulas of sort `Formula` are formalized as follows.

```
sort Formula .
op P : Nat Vect     -> Formula [ctor] .
op P : Nat Nat Vect -> Formula [ctor] .
op _and_ : Formula Formula -> Formula [comm assoc prec 55] .
```

We formalize projection operators of the form $P(q_i, |\psi\rangle)$ and $\mathrm{P}(q_i, q_j, |\psi'\rangle)$ using the operators `P(_,_)` and `P(_,_,_)`, respectively, to check whether the final quantum state in a computation belongs to a closed subspace spanned by $\{|\psi\rangle\}$ at qubit q_i and by $\{|\psi'\rangle\}$ at qubits q_i and q_j. We also formalize the conjunction of formulas using the operator `_and_`.

To use the algebraic specification `|AS4QC>`, we only need to import the functional module `QC`, after which all facilities for reasoning about quantum computation become readily available.

4 A Case Study: Quantum Teleportation in |AS4QC>

This section presents how to formally specify and verify Quantum Teleportation in `|AS4QC>` as a case study. Quantum Teleportation [1] is a quantum communication protocol that enables the teleportation of an arbitrary single-qubit state $|\psi\rangle$ from Alice to Bob. The quantum circuit for this protocol is shown in Figure 2. The protocol begins with the initial quantum state $|\psi\rangle_{q_0} \otimes |0\rangle_{q_1} \otimes |0\rangle_{q_2}$. It first applies the H gate to q_1, followed by the CX gate on q_1 and q_2 to create an entangled state shared by Alice and Bob. Alice then applies the CX gate to q_0 and q_1, followed by the H gate on q_0. Alice then measures q_2 and q_1 in sequence

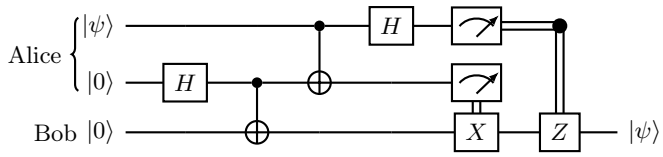


Fig. 2: Quantum Teleportation

and sends the measurement outcomes to Bob. Based on these outcomes, Bob conditionally applies the X and Z gates to recover the state $|\psi\rangle$ on qubit q_2. We want to verify whether $|\psi\rangle$ has been successfully teleported with certainty.

We first specify the functional module `TELEPORT` in `|AS4QC>` for Quantum Teleportation as follows:

```
fmod TELEPORT is
  pr QC .
  ops a b  : -> NzCpx .
  op |psi> : -> Vect  .
  eq |psi> = a . |0> + b . |1> .
  eq  a * a ^* + b * b ^* = (1,0,0,0) .
endfm
```

We define two fresh constants (or symbolic values), `a` and `b`, of sort `NzCpx`, representing nonzero complex numbers. We then define a symbolic quantum state `|psi>` of sort `Vect` as `a.|0> + b.|1>`, where `a` and `b` are subject to the normalization constraint shown in the last equation.

To verify the correctness of Quantum Teleportation, we must consider all possible measurement outcomes that can occur in the protocol. Therefore, we prepare the following command to verify the correctness of the protocol and then ask Maude to reduce it.

```
red analyze([
  H(1) CX(1,2) CX(0, 1) H(0)
  (P0(1) U (P1(1) X(2))) (P0(0) U (P1(0) Z(2))),
  |psi> (x) |0> (x) |0>]) .
```

The list of quantum operations presents the behavior of the protocol where the nondeterministic choices `_U_` and the projection operators `P0` and `P1` for measurement outcomes 0 and 1, respectively, are used to handle the nondeterminism of measurement. The initial quantum state is `|psi>(x)|0>(x)|0>`. Maude completes the command shortly and returns the following result.

```
[|0>(x)|0>(x)(a . |0> + b . |1>), 2.5e-1],
[|1>(x)|0>(x)(a . |0> + b . |1>), 2.5e-1],
[|0>(x)|1>(x)(a . |0> + b . |1>), 2.5e-1],
[|1>(x)|1>(x)(a . |0> + b . |1>), 2.5e-1]
```

The result shows that each computation consists of a final quantum state whose third qubit is `|psi>`, with a reaching probability of 1/4. Consequently, the total

probability of reaching a final quantum state whose third qubit is `|psi>` is 1. We can also use the following command to check the correctness of the protocol.

```
red check([
  H(1) CX(1,2) CX(0, 1) H(0)
  (P0(1) U (P1(1) X(2))) (P0(0) U (P1(0) Z(2))),
  |psi> (x) |0> (x) |0>], P(2,|psi>)) .
```

Maude returns an empty set of computations, meaning that all final computations have a final quantum state whose third qubit is `|psi>`. Therefore, we can conclude the correctness of Quantum Teleportation. Note that we can perform the same analysis when `|psi>` is initialized to either `a.|0>` or `b.|1>` to cover the remaining cases of the quantum state $|\psi\rangle$, and we can obtain the expected result. Using the algebraic specification `|AS4QC>`, we can represent complex numbers symbolically, which allows us to specify arbitrary quantum states for symbolic reasoning. This capability is a major advantage of our approach.

5 Experiments

This section presents the experimental results using `|AS4QC>` to verify the correctness of several case studies. The experiments were conducted using Maude 3.5 on a MacBook Pro machine equipped with an Apple M4 Max chip and 48 GB of RAM. `|AS4QC>` is available at https://github.com/canhminhdo/as4qc.

In the sequel, we present several case studies and their desired properties that we want to verify to demonstrate the effectiveness of our algebraic specification `|AS4QC>` for modeling, reasoning about, and verifying quantum programs.

- Entanglement for constructing an entangled state with multiple qubits.
- Quantum Teleportation [1] for teleporting an arbitrary qubit state $|\psi\rangle$ from a sender to a receiver. Starting from the initial quantum state $|\psi\rangle_{q_0} \otimes |0\rangle_{q_1} \otimes |0\rangle_{q_2}$, we verify whether $|\psi\rangle$ is teleported successfully to qubit q_2 with certainty at the final quantum state.
- Entanglement Swapping [28] for creating a new entangled state $|\Phi^+\rangle$. Given the initial quantum state $|0\rangle_{q_0} \otimes |0\rangle_{q_1} \otimes |0\rangle_{q_2} \otimes |0\rangle_{q_3}$, we verify whether the final joint state of qubits q_0 and q_3 is the EPR state $|\Phi^+\rangle$ at the end.
- Quantum Secret Sharing [14] for teleporting an arbitrary qubit state $|\psi\rangle$ from a sender to a receiver with the help of a third party. From the initial quantum state $|\psi\rangle_{q_0} \otimes |0\rangle_{q_1} \otimes |0\rangle_{q_2} \otimes |0\rangle_{q_3}$, we verify whether $|\psi\rangle$ is teleported successfully to qubit q_3 with certainty at the final quantum state.
- Quantum Relay Scheme [4] for teleporting an arbitrary quantum state $|\psi\rangle$ from one quantum device to another wirelessly, even when these two devices do not share EPR pairs mutually. Starting with the initial quantum state $|\psi\rangle_{q_0} \otimes |0\rangle_{q_1} \otimes |0\rangle_{q_2} \otimes |0\rangle_{q_3} \otimes |0\rangle_{q_4}$, we verify whether $|\psi\rangle$ is teleported successfully to qubit q_4 with certainty at the final quantum state.
- Bidirectional Quantum Teleportation [13] based on EPR pairs and entanglement swapping for simultaneously transmitting arbitrary qubit states $|\psi\rangle$ and $|\phi\rangle$ between two users. From the initial quantum state $|\psi\rangle_{q_0} \otimes |\phi\rangle_{q_1} \otimes$

Table 2: Experimental results with `|AS4QC>`

Program	**Qubits**	**Unitary**	**Measurement**	**Time**
Entanglement	100	100	0	53ms
Quantum Teleportation	3	8	2	≈ 0
Entanglement Swapping	4	8	2	≈ 0
Quantum Secret Sharing	4	9	3	≈ 0
Quantum Relay Scheme	5	12	4	1ms
Bidirectional Teleportation	6	12	4	1ms
Quantum Network Coding	14	37	10	268ms
Grover's Search	5	133	0	10ms
	10	1,430	0	1.2s
	15	11,973	0	3.9m

$|0\rangle_{q_2} \otimes |0\rangle_{q_3} \otimes |0\rangle_{q_4} \otimes |0\rangle_{q_5}$, we verify whether $|\psi\rangle$ and $|\phi\rangle$ are simultaneously teleported in qubits q_3 and q_4 with certainty at the final quantum state.

- Quantum Network Coding [21] for the simultaneous generation of EPR pairs using only local operations and classical communication with EPR pairs shared between adjacent quantum repeaters. Given the initial quantum state $|0\rangle_{q_0} \otimes \cdots \otimes |0\rangle_{q_{13}}$, we verify whether the final quantum state contains EPR pairs $|\Phi^+\rangle$ between qubits (q_1, q_4) and (q_0, q_5) at the end.
- Grover's search algorithm [12] for searching an unstructured database with a quadratic speedup by iteratively applying the Grover operator. We verify whether, after approximately $\pi\sqrt{2^n}/4$ iterations of the Grover operator, the correct solution is obtained with high probability, where n and 2^n denote the number of qubits and the database size, respectively.

We successfully verified the correctness of all case studies against their desired properties using `|AS4QC>`. The experimental results are summarized in Table 2. The columns **Qubits**, **Unitary**, **Measurement** denote the numbers of qubits, unitary operations, and measurements, respectively, while the column **Time** reports the time taken by Maude to verify each program against its specified property. For the Grover's Search case study, measurements are not required because we can directly query the amplitude of the correct solution in the final quantum state supported by `|AS4QC>`, from which the probability of reaching the correct solution can be computed. Consequently, the number of measurements in Grover's Search is zero. For all case studies except Grover's Search with 15 qubits, we could complete the verification tasks quickly. Especially, we could construct an entangled state with 100 qubits in just 53 milliseconds, comparable with graph-based quantum simulators such as QuIDDPro [26] and MQT Core

DD [30], while array-based quantum simulators such as LIQUi$|\rangle$ [27], QX [15], and ProjectQ [25] could not do so as reported in [30]. In the case of Grover's Search with 15 qubits, the verification involves a nontrivial workload with 11,973 applications of quantum operations. As a result, Maude took approximately 3.9 minutes to complete the verification. Notably, `|AS4QC>` is capable of handling not only concrete and exact values, as presented in Entanglement, Entanglement Swapping, Quantum Network Coding, and Grover's Search, but also symbolic values, as demonstrated in the other case studies. These experimental results demonstrate the effectiveness and practicality of `|AS4QC>` for modeling, symbolic and exact reasoning about, and verifying quantum programs.

6 Related Work and Conclusion

Symbolic approach to reasoning about quantum computation has been used for theorem proving of quantum circuits in Coq [23] and for reachability analysis, model checking, and equivalence checking of quantum circuits in Maude [8,10,9,7]. However, these approaches can handle simple case studies and a limited number of qubits, mainly due to difficulties in representing complex numbers symbolically and quantum states effectively. Graph-based (or decision diagram-based) approaches [26,30] significantly outperform array-based approaches [27,15,25] for quantum simulation by compactly representing quantum states, exploiting structural redundancies, and providing efficient algorithms for quantum operations. The use of the ring $\mathbb{D}[\omega]$ in decision diagrams to enable symbolic and exact quantum computation was preliminarily investigated in [29] and has since been adopted for the formal verification of quantum circuits and quantum programs in [3,2]. To the best of our knowledge, there is no existing algebraic specification that uses the ring $\mathbb{D}[\omega]$ for symbolic reasoning about quantum computation. Meanwhile, algebraic approaches are expressive and powerful for symbolic reasoning. Thus, `|AS4QC>` provides a promising algebraic framework for symbolic and exact reasoning about quantum computation in Maude, upon which verification techniques for quantum systems in Maude can be developed in the future.

We have presented `|AS4QC>`, an algebraic specification for quantum computation in Maude. We have employed the ring $\mathbb{D}[\omega]$ to represent complex numbers in quantum computation without approximation, used Dirac notation to model quantum states and quantum operations, and leveraged a set of laws from quantum mechanics and basis matrix operations to automate reasoning about quantum computation. As case studies, we have used `|AS4QC>` to confirm the correctness of several nontrivial quantum programs, demonstrating the effectiveness and practicality of our approach. Although a complex number can be algebraically represented by a quadruple (a, b, c, d) and a normalization factor k, many different choices of (a, b, c, d) and k may represent the same complex number. To use model checking techniques effectively, it is therefore necessary to identify a unique representation for this algebraic form. Developing such a unique representation, as suggested in [29], would be one part of our future work, in addition to conducting more complex case studies, such as Shor's algorithm.

References

1. Bennett, C.H., Brassard, G., Crépeau, C., Jozsa, R., Peres, A., Wootters, W.K.: Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. Phys. Rev. Lett. **70**, 1895–1899 (1993). https://doi.org/10.1103/PhysRevLett.70.1895
2. Chen, Y.F., Chung, K.M., Hsieh, M.H., Huang, W.J., Lengál, O., Lin, J.A., Tsai, W.L.: AutoQ 2.0: From verification of quantum circuits to verification of quantum programs. In: Tools and Algorithms for the Construction and Analysis of Systems. pp. 87–108. Springer Nature Switzerland (2025). https://doi.org/10.1007/978-3-031-90660-2_5
3. Chen, Y.F., Chung, K.M., Lengál, O., Lin, J.A., Tsai, W.L.: AutoQ: An automata-based quantum circuit verifier. In: Computer Aided Verification. pp. 139–153. Springer Nature Switzerland (2023). https://doi.org/10.1007/978-3-031-37709-9_7
4. Cheng, S.T., Wang, C.Y., Tao, M.H.: Quantum communication for wireless wide-area networks. IEEE Journal on Selected Areas in Communications **23**(7), 1424–1432 (2005). https://doi.org/10.1109/JSAC.2005.851157
5. Dirac, P.A.M.: A new notation for quantum mechanics. Mathematical Proceedings of the Cambridge Philosophical Society **35**(3), 416–418 (1939). https://doi.org/10.1017/S0305004100021162
6. Do, C.M., Ogata, K.: Symbolic model checking quantum circuits in Maude. In: The 35th International Conference on Software Engineering and Knowledge Engineering, SEKE 2023. pp. 103–108 (2023). https://doi.org/10.18293/SEKE2023-014
7. Do, C.M., Ogata, K.: Equivalence checking of quantum circuits based on Dirac notation in Maude. In: Rewriting Logic and Its Applications. pp. 84–103. Springer (2024). https://doi.org/10.1007/978-3-031-65941-6_5
8. Do, C.M., Ogata, K.: An executable operational semantics of quantum programs and its application. In: Software Fault Prevention, Verification, and Validation. pp. 15–31. Springer Nature Singapore (2024). https://doi.org/0.1007/978-981-96-1621-3_2
9. Do, C.M., Ogata, K.: Symbolic model checking quantum circuits in Maude. PeerJ Comput. Sci. **10**, e2098 (2024). https://doi.org/10.7717/PEERJ-CS.2098
10. Do, C.M., Takagi, T., Ogata, K.: Automated quantum protocol verification based on Concurrent Dynamic Quantum Logic. ACM Trans. Softw. Eng. Methodol. **34**(6) (Jul 2025). https://doi.org/10.1145/3708475
11. Giles, B., Selinger, P.: Exact synthesis of multiqubit Clifford+T circuits. Phys. Rev. A **87**, 032332 (Mar 2013). https://doi.org/10.1103/PhysRevA.87.032332
12. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing. p. 212–219. STOC '96, Association for Computing Machinery (1996). https://doi.org/10.1145/237814.237866
13. Hassanpour, S., Houshmand, M.: Bidirectional quantum teleportation and secure direct communication via entanglement swapping (2014). https://doi.org/10.48550/arXiv.1411.0206
14. Hillery, M., Bužek, V., Berthiaume, A.: Quantum secret sharing. Phys. Rev. A **59**, 1829–1834 (1999). https://doi.org/10.1103/PhysRevA.59.1829
15. Khammassi, N., Ashraf, I., Fu, X., Almudever, C., Bertels, K.: QX: A high-performance quantum computer simulation platform. In: Design, Automation &

Test in Europe Conference & Exhibition (DATE). pp. 464–469 (2017). https://doi.org/10.23919/DATE.2017.7927034
16. Kliuchnikov, V., Maslov, D., Mosca, M.: Fast and efficient exact synthesis of single-qubit unitaries generated by Clifford and T gates. Quantum Info. Comput. **13**(7–8), 607–630 (Jul 2013). https://doi.org/10.26421/QIC13.7-8-4
17. M. Clavel, et al.: All About Maude – A High-Performance Logical Framework: How to Specify, Program and Verify Systems in Rewriting Logic, Lecture Notes in Computer Science, vol. 4350. Springer (2007). https://doi.org/10.1007/978-3-540-71999-1
18. Meseguer, J.: Twenty Years of Rewriting Logic. J. Log. Algebraic Methods Program. **81**(7-8), 721–781 (2012). https://doi.org/10.1016/j.jlap.2012.06.003
19. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge University Press (2010). https://doi.org/10.1017/CBO9780511976667
20. Paykin, J., Rand, R., Zdancewic, S.: QWIRE: A core language for quantum circuits. SIGPLAN Not. **52**(1), 846–858 (jan 2017). https://doi.org/10.1145/3093333.3009894
21. Satoh, T., Gall, F.L., Imai, H.: Quantum network coding for quantum repeaters. Physical Review A **86**(3) (sep 2012). https://doi.org/10.1103/physreva.86.032331
22. Selinger, P.: Towards a quantum programming language. Mathematical. Structures in Comp. Sci. **14**(4), 527–586 (aug 2004). https://doi.org/10.1017/S0960129504004256
23. Shi, W., Cao, Q., Deng, Y., Jiang, H., Feng, Y.: Symbolic reasoning about quantum circuits in Coq. J. Comput. Sci. Technol. **36**(6), 1291–1306 (2021). https://doi.org/10.1007/s11390-021-1637-9
24. Shor, P.: Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings 35th Annual Symposium on Foundations of Computer Science. pp. 124–134 (1994). https://doi.org/10.1109/SFCS.1994.365700
25. Steiger, D.S., Häner, T., Troyer, M.: ProjectQ: an open source software framework for quantum computing. Quantum **2**, 49 (Jan 2018). https://doi.org/10.22331/q-2018-01-31-49
26. Viamontes, G., Markov, I., Hayes, J.: High-performance QuIDD-based simulation of quantum circuits. In: Proceedings Design, Automation and Test in Europe Conference and Exhibition. vol. 2, pp. 1354–1355 Vol.2 (2004). https://doi.org/10.1109/DATE.2004.1269084
27. Wecker, D., Svore, K.M.: LIQUi|>: A software design architecture and domain-specific language for quantum computing (2014). https://doi.org/10.48550/arXiv.1402.4467
28. Żukowski, M., Zeilinger, A., Horne, M.A., Ekert, A.K.: "Event-ready-detectors" Bell experiment via entanglement swapping. Phys. Rev. Lett. **71**, 4287–4290 (1993). https://doi.org/10.1103/PhysRevLett.71.4287
29. Zulehner, A., Niemann, P., Drechsler, R., Wille, R.: Accuracy and compactness in decision diagrams for quantum computation. In: 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE). pp. 280–283 (2019). https://doi.org/10.23919/DATE.2019.8715040
30. Zulehner, A., Wille, R.: Advanced simulation of quantum computations. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **38**(5), 848–859 (2019). https://doi.org/10.1109/TCAD.2018.2834427